

Feature Summary: Likes and Comments for Community Posts

Overview

This feature adds interactive like and comment functionality to community posts in The Writer's Corner application, enabling users to engage with each other's exercise submissions.

Features Implemented

1. Like Functionality

- ☒ Users can like/unlike posts with a single click
- ☒ Like count is displayed on each post
- ☒ Visual indication (filled heart icon) when current user has liked a post
- ☒ Prevents duplicate likes (enforced at database level)
- ☒ Real-time UI updates without page refresh

2. Comment Functionality

- ☒ Users can add comments to posts
- ☒ All comments are displayed with author information and timestamp
- ☒ Users can edit their own comments
- ☒ Users can delete their own comments (with confirmation)
- ☒ Comments show "(edited)" label if modified
- ☒ Expandable/collapsible comment section per post
- ☒ Comment count displayed on each post

Technical Implementation

Database Changes

New Models:

- `SubmissionLike` - Tracks likes on exercise submissions
- `SubmissionComment` - Tracks comments on exercise submissions
- `PostLike` - Tracks likes on community posts (future-proof)
- `PostComment` - Tracks comments on community posts (future-proof)

Updated Models:

- `User` - Added relations for likes and comments
- `ExerciseSubmission` - Added relations for likes and comments
- `CommunityPost` - Added relations for likes and comments (future-proof)

API Endpoints Created

Likes API (`/api/community/submissions/likes`):

- `POST` - Toggle like/unlike on a submission
- `GET` - Get like information for a submission

Comments API (/api/community/submissions/comments):

- POST - Add a new comment to a submission
- GET - Get all comments for a submission
- PUT - Update a comment (user's own only)
- DELETE - Delete a comment (user's own only)

Updated API (/api/community/posts):

- Enhanced to include like count, comment count, and user's like status

Frontend Changes

Component: `community-overview.tsx`

- Added state management for likes and comments
- Implemented interactive like button with visual feedback
- Created expandable comment section with:
 - Comment input form
 - Comment list with author info
 - Edit/delete controls for own comments
- Real-time updates
- Maintained existing vintage/parchment theme styling

Security & Validation

-  All API endpoints require authentication
-  Users can only edit/delete their own comments
-  Posts must be public to receive likes/comments
-  Comment content validation (non-empty)
-  Database-level unique constraints prevent duplicate likes

User Experience

1. Like a Post:

- Click the heart icon to like
- Icon fills with color and count updates
- Click again to unlike

2. View Comments:

- Click "Comment" button to expand comment section
- See all existing comments with timestamps

3. Add a Comment:

- Type in the comment input field
- Press Enter or click "Post" button
- Comment appears immediately in the list

4. Edit a Comment:

- Click "Edit" on your own comment
- Modify the text inline
- Click "Save" to update or "Cancel" to discard

5. Delete a Comment:

- Click "Delete" on your own comment

- Confirm in the dialog
- Comment is removed immediately

Code Quality

- TypeScript types defined for all data structures
- Error handling in all API routes
- Consistent code style matching existing patterns
- Reusable UI components from Radix UI library
- Optimistic UI updates for better user experience

Testing Checklist

Backend Testing

- ☐ Like a post successfully
- ☐ Unlike a post successfully
- ☐ Prevent duplicate likes (database constraint)
- ☐ Add a comment successfully
- ☐ Edit own comment successfully
- ☐ Delete own comment successfully
- ☐ Prevent editing others' comments (403 error)
- ☐ Prevent deleting others' comments (403 error)
- ☐ Handle non-existent submissions (404 error)
- ☐ Handle non-public submissions (403 error)
- ☐ Require authentication for all operations (401 error)

Frontend Testing

- ☐ Like button updates immediately on click
- ☐ Like count displays correctly
- ☐ Filled heart icon shows for liked posts
- ☐ Comment section expands/collapses
- ☐ Comments load when expanding section
- ☐ Add comment updates UI immediately
- ☐ Edit comment shows inline editor
- ☐ Save edited comment updates display
- ☐ Cancel editing discards changes
- ☐ Delete comment removes from list
- ☐ Confirmation dialog appears on delete
- ☐ Only show edit/delete for own comments
- ☐ Display "(edited)" label for modified comments
- ☐ Timestamps format correctly

Integration Testing

- ☐ Multiple users can like the same post
- ☐ Like count increments/decrements correctly
- ☐ Comments from different users display correctly

- [] Editing one comment doesn't affect others
- [] Deleting a post cascades to likes/comments
- [] User deletion cascades to their likes/comments

Future Enhancements

Potential improvements for future iterations:

- Nested/threaded comments (replies to comments)
- Notification system for likes and comments
- Markdown support in comments
- Comment pagination for posts with many comments
- Like button animation
- Comment sorting options (newest/oldest)
- Rich text editor for comments
- @ mentions in comments

Files Modified

1. `app/prisma/schema.prisma` - Database schema updates
2. `app/app/api/community/posts/route.ts` - Enhanced to include like/comment data
3. `app/app/api/community/submissions/likes/route.ts` - New like API
4. `app/app/api/community/submissions/comments/route.ts` - New comment API
5. `app/components/community/community-overview.tsx` - Frontend implementation

Files Added

1. `DATABASE_MIGRATION.md` - Migration documentation
2. `FEATURE_SUMMARY.md` - This file

Migration Instructions

After pulling this PR:

```
cd app

# Install dependencies if needed
npm install

# Generate Prisma client
npx prisma generate

# Apply database migration
npx prisma migrate dev --name add_likes_and_comments

# Or push to database directly
npx prisma db push

# Start development server
npm run dev
```

Notes for Reviewers

- All features follow existing code patterns and conventions
- UI maintains the vintage/parchment theme of the application
- Database schema supports both ExerciseSubmission and CommunityPost for future flexibility
- API routes include comprehensive error handling and validation
- Frontend provides optimistic updates for smooth user experience
- Security is enforced at both API and database levels